Day 145 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 6 reference

Watch: Ethical Concerns with Generative AI (Part 1 of 2) (Douglas Schmidt) | How to spot deepfakes and AI-generated images (New Scientist)

`day145_generative_ai_ethics.py`, segment by segment, in the order `main()` uses things. Skewing a training set, the detector, its probability, the blur and the base-rate formula are new; the networks (Day 126), VAE (Days 129–130), GAN (Day 134), scorer (Day 138), diffusion model (Day 141), DDIM (Day 142), encoders and measurements (Day 143) and the text-conditioned model with guidance (Day 144) are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: TPR, FPR = true and false positive rates · p = prevalence (share of images that are generated) · w = guidance scale

Segment 0 — Setup

Imports: nine earlier parts

CODE

```
sys.path.insert(0, HERE)
for w in ("week-20-generative-models-gans", "week-19-unsupervised-learning-ii",
    "week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day129_exercise as d129
    import day130_exercise as d130
    import day134_exercise as d134
    import day139_gan_build_and_forward_diffusion as d139
    import day141_exercise as d141
    import day142_exercise as d142
    import day143_exercise as d143
    import day144_exercise as d144
```

- `import day144_exercise as d144` — importing an exercise runs its own checks once; redirect_stdout hides what they print.
- `for w in (...): sys.path.insert(0, ...)` — earlier weeks' folders added to the search path, so their files import by name.

Constants

CODE

```
SKEW = 0.75
MEM_SIZES = (10, 100, 1000)
MEM_STEPS = 8000
NEAR = 2.0
```

- `MEM_SIZES = (10, 100, 1000)` — a tuple: the loop in section 4 trains one model per size.
- `NEAR = 2.0` — a pixel distance; Day 141's model trained on all 60,000 had its closest sample 1.37 away.

Segment 1 — The new parts

skew_ones: removing ones at random

MATH

$$\text{kept others} = r\,(1 - s)\,/\,s \quad \Rightarrow \quad r\,/\,(r + \text{kept others}) = s$$

CODE

```
def skew_ones(y, lean, share=SKEW, seed=0):
    rng = np.random.RandomState(seed); idx = np.arange(len(y))
    one = y == 1; right = one & (lean == 1); other = one & (lean != 1)
    keep_other = int(right.sum() * (1 - share) / share)
    drop = rng.permutation(idx[other])[keep_other:]
    return np.setdiff1d(idx, drop)
```

- `one & (lean == 1)` — & combines two boolean arrays element by element: ones that lean right.
- `int(right.sum() * (1 - share) / share)` — if r ones lean right, keeping r(1 – s)/s others makes the share leaning right s.
- `rng.permutation(idx[other])[keep_other:]` — the other ones in random order; everything after the first keep_other is dropped.
- `np.setdiff1d(idx, drop)` — every index not in drop, sorted.

train_detector and p_fake: a small network with a logistic loss

MATH

$$p = \sigma(\text{score}) = 1\,/\,(1 + e^{-\text{score}}), \quad \text{loss} = -[y \log p + (1 - y) \log(1 - p)], \quad d\,\text{loss}\,/\,d\,\text{score} = p - y$$

CODE

```
def train_detector(R, G, epochs=10, seed=0, batch=128, lr=1e-3):
    net = d126.init_net([784, 128, 1], seed); opt = d126.Adam(net, lr); rng =
        np.random.RandomState(seed)
    Xd = np.vstack([R, G]).astype(np.float32); yd = np.r_[np.zeros(len(R)), np.ones(len(G))]
    for _ in range(epochs):
        o = rng.permutation(len(Xd))
        for s in range(0, len(Xd) - batch + 1, batch):
            b = o[s:s + batch]; A = d126.forward(net, Xd[b], ["relu", "linear"])
            p = 1 / (1 + np.exp(-A[-1][:, 0].astype(np.float64)))
            grads, _ = d126.backward(net, A, ["relu", "linear"], ((p - yd[b]) / len(b))[:,
                None].astype(np.float32))
            opt.step(net, grads)
    return net
def p_fake(net, F):
    return 1 / (1 + np.exp(-d126.forward(net, F.astype(np.float32), ["relu", "linear"])[-1][:,
        0].astype(np.float64)))
```

- `np.r_[np.zeros(len(R)), np.ones(len(G))]` — np.r_ joins arrays end to end: label 0 for each real image, 1 for each generated one.
- `A[-1][:, 0]` — the output layer has one unit; [:, 0] turns the n × 1 result into n numbers.
- `1 / (1 + np.exp(-...))` — the sigmoid: any score becomes a probability between 0 and 1.
- `((p - yd[b]) / len(b))[:, None]` — the logistic loss's gradient with respect to the score, averaged over the batch, as a column.

blur and flagged_precision

MATH

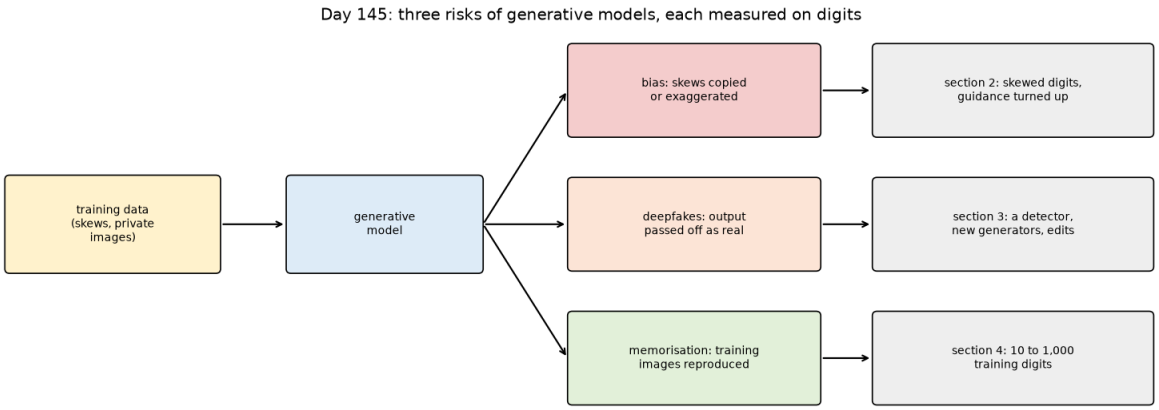
$$\text{flagged that are generated} = TPR \cdot p \,/\, (TPR \cdot p + FPR \cdot (1 - p))$$

CODE

```
def blur(F):
    P = np.pad(F.reshape(-1, 28, 28), ((0, 0), (1, 1), (1, 1)))
    return (sum(P[:, i:i + 28, j:j + 28] for i in range(3) for j in range(3)) / 9).reshape(-1,
        784)
def flagged_precision(tpr, fpr, prevalence):
    return tpr * prevalence / (tpr * prevalence + fpr * (1 - prevalence))
```

- `np.pad(F.reshape(-1, 28, 28), ((0, 0), (1, 1), (1, 1)))` — a border of zeros one pixel wide around each image, none along the first axis.
- `sum(P[:, i:i + 28, j:j + 28] for i in range(3) for j in range(3))` — nine shifted copies of every image added up: each pixel gets its 3 × 3 neighbourhood.
- `return tpr * prevalence / (...)` — Bayes' rule, written once and used in sections 1 and 3.

SYSTEM DIAGRAM



Three risks, each measured.

Segment 2 — The experiments

demo_hand: one formula, four prevalences

CODE

```
for p in (0.5, 0.1, 0.01, 0.001):
    f = flagged_precision(0.95, 0.05, p)
    print(f"{p:8.3f} {f:27.3f} {0.05 * (1 - p) / (0.95 * p):45.2f}")
print(f"by hand for p = 0.01: 0.95 x 0.01 = 0.0095; 0.05 x 0.99 = 0.0495; 0.0095 / (0.0095 +
    0.0495) = {0.0095 / 0.059:.3f}")
```

- `0.05 * (1 - p) / (0.95 * p)` — real images wrongly flagged for each generated image caught.
- `{0.0095 / 0.059:.3f}` — the hand calculation, computed inside the f-string.

demo_bias: an inner function used twice

CODE

```
def table(ema, txt, prompts, tag):
    rows, pics = [], {}
    for prompt, d in prompts:
        for w in (1.0, 2.0, 4.0):
            F = d144.cfg_sample(ema, d143.embed_texts(txt, [prompt.split()])[0], 400, w); lab =
                d144.said(sc, F)
            ik, ln = d144.ink_lean(F); k = lab == d
            r, l_, th = np.mean(ln[k] > cut_lean[d][1]), np.mean(ln[k] < cut_lean[d][0]),
                np.mean(ik[k] > cut_ink[d])
            rows.append((prompt + tag, w, r, th)); pics[(prompt + tag, w)] = F[:12]
            print(f"{prompt + tag:>22} {w:5.1f} {k.mean():15.3f} {r:14.3f} {l_:13.3f} {th:7.3f}")
    return rows, pics
```

- `def table(ema, txt, prompts, tag):` — defined inside demo_bias, so it can use sc, cut_lean and cut_ink from there; called once for the skewed model and once for the control.
- `[prompt.split()]` — the prompt as a list of words, inside a list: one caption.
- `k = lab == d` — a boolean mask: only samples named as the asked-for digit are measured.
- `np.mean(ln[k] > cut_lean[d][1])` — the share of those whose lean is past that digit's upper cut point (leaning right).
- `pics[(prompt + tag, w)] = F[:12]` — a dictionary keyed by a tuple: prompt and guidance scale.

demo_bias: the control

CODE

```
t0 = time.perf_counter()
_, txt0, _ = d143.train_clip(X[:60000], y[:60000], thick[:60000], lean[:60000])
_, ema0, _ = d144.train_cond(X2[:60000], y[:60000], thick[:60000], lean[:60000], txt0)
print(f"\ncontrol: the same encoders and model, same seeds, trained on all 60,000 training digits
    (no skew): {time.perf_counter() - t0:.0f} s")
r0, p0 = table(ema0, txt0, (("one", 1), ("seven", 7)), " (no skew)"); rows += r0; pics.update(p0)
```

- `X[:60000], y[:60000], ...` — the same calls as for the skewed set, on all 60,000 training digits; the seeds are the functions' defaults, so they match.
- `rows += r0; pics.update(p0)` — += extends the list in place; update adds the control's entries to the dictionary.

demo_detector: four edits as lambdas

CODE

```
edits = (("3 x 3 blur", blur), ("noise, sd 0.1", lambda F: np.clip(F +
    np.random.RandomState(5).normal(0, 0.1, F.shape), 0, 1)),
         ("4 grey levels", lambda F: np.round(F * 3) / 3), ("black and white (0.5)", lambda F: (F
             > 0.5).astype(np.float32)))
for name, f in edits:
    for who, F in (("generated", Gt), ("real", Rt)):
        res[f"{who}, {name}"] = float(np.mean(p_fake(det, f(F)) > 0.5)); print(f"{who + ', ' +
            name:>44} {res[who + ', ' + name]:17.3f}")
```

- `lambda F: np.clip(...)` — a one-line function with no name; each edit is a (name, function) pair.
- `np.round(F * 3) / 3` — every pixel rounded to 0, 1/3, 2/3 or 1: four grey levels.
- `(F > 0.5).astype(np.float32)` — True/False turned into 1.0/0.0: black and white.
- `for who, F in (("generated", Gt), ("real", Rt))` — each edit applied to both held-out sets.

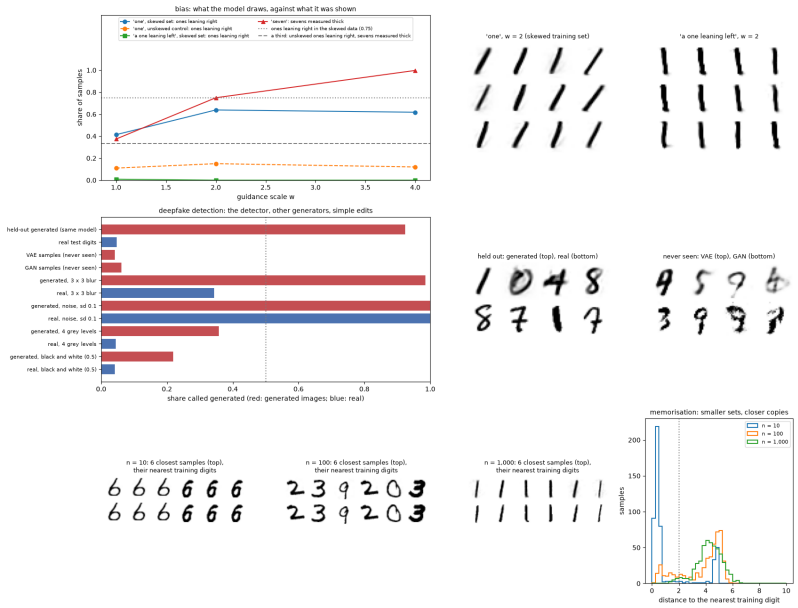
demo_memorise: one model per training-set size

CODE

```
for n in MEM_SIZES:
    t0 = time.perf_counter(); idx = order[:n]
    net, _ = d141.train_image_denoiser(X2[idx], MEM_STEPS)
    F = d141.to_pixels(d142.ddim_sample(d142.net_x0_fn(net), 500, np.random.RandomState(1), 100,
        0.0))
    dF, jF = d141.nearest_train(F, X[idx]); dT, _ = d141.nearest_train(Te, X[idx])
    out[n] = (F, X[idx][jF], dF)
```

- `idx = order[:n]` — the first n of one fixed shuffle, so the 100 include the 10 and the 1,000 include the 100.
- `d141.nearest_train(F, X[idx])` — Day 141's helper: for each sample, the distance to and index of its nearest training digit.
- `X[idx][jF]` — the nearest training digit of each sample, for the figure.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `a & b (boolean arrays)` — element-wise and.
- `np.r_[a, b]` — join arrays end to end.
- `np.setdiff1d(a, b)` — the values of a not in b, sorted.
- `np.pad(A, ((0, 0), (1, 1), (1, 1)))` — a border of zeros on chosen axes.
- `sum(generator)` — add up arrays produced one at a time.
- `lambda F: ...` — a one-line function with no name.
- `def inner(): ... inside a function` — a helper that uses the outer function's names.
- `d.update(other)` — add another dictionary's entries.

End of Day 145 syntax reference. For the experiments and what they showed, see the Day 145 complete guide.